

General Purpose Input Output

Introduction to ARM Cortex-M Microprocessors – Chapter 8

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

1 Why This Matters

2 Inside a GPIO Pin

3 The GPIO Registers

4 Experiments

5 Design Challenge

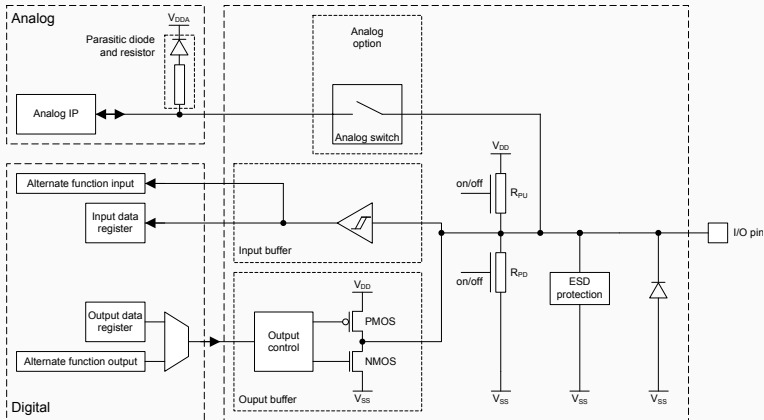
- 1** Why This Matters
- 2 Inside a GPIO Pin
- 3 The GPIO Registers
- 4 Experiments
- 5 Design Challenge

Chapter 4 already blinked an LED in C. Chapters 5–7 then opened up what that program became underneath — the load, the mask, the store, the branch. This chapter closes the loop: the same MODER/ODR pattern you have now seen at both levels becomes the foundation for every peripheral in the rest of this book. A GPIO pin is the simplest bridge between software and the physical world, so it is where that pattern is easiest to see start to finish.

After completing this chapter, you should be able to:

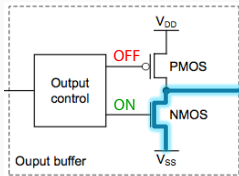
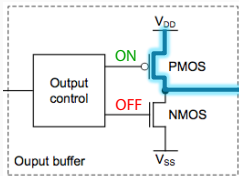
1. Describe the parts inside a GPIO pin: the Schmitt trigger, the output driver, and the pull resistors.
2. Choose between push-pull and open-drain outputs for a given external circuit.
3. Configure any pin's mode, output type, speed, and pull resistors using the GPIO registers.
4. Explain why a peripheral clock must be enabled before its registers respond.
5. Use BSRR for atomic pin changes, and explain why it beats a read-modify-write on ODR.
6. Read a switch input reliably, including software debouncing.
7. Drive a multi-segment display from a lookup table instead of per-segment logic.

- 1 Why This Matters
- 2 **Inside a GPIO Pin**
- 3 The GPIO Registers
- 4 Experiments
- 5 Design Challenge

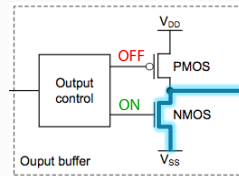
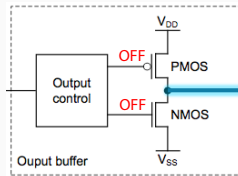


One pin has two separate paths: an **input** path through a Schmitt trigger to IDR, and an **output** path from ODR through the driver transistors. Pull resistors sit in between.

Push-Pull and Open-Drive



Push-pull — drives both high and low

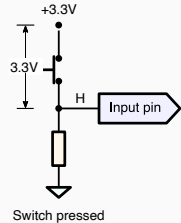
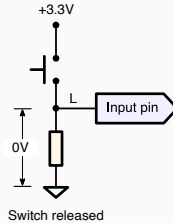
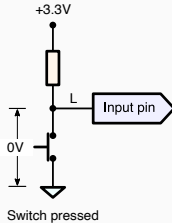
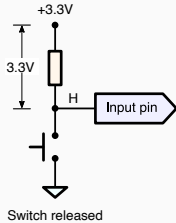


Open-drain — pulls low, or releases

When you need open-drain

Push-pull is the default. It is what you want for an LED. But if two devices share one wire, one driving high while the other drives low causes a **short circuit**. Open-drain lets many devices share one line safely. This is exactly how I²C works (Chapter 13).

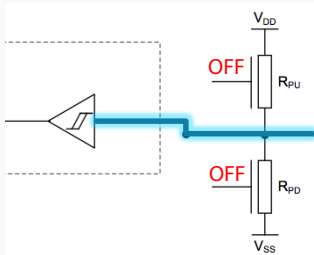
Pull-Up and Pull-Down



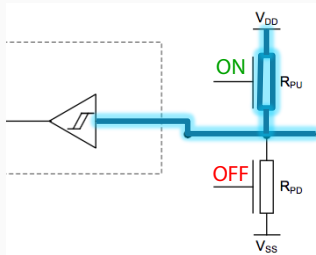
An unconnected input *floats*

It picks up noise and reads a random value. A pull resistor gives it a fixed level when nothing else is driving it.

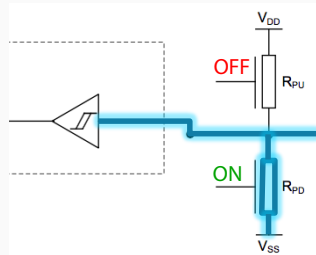
A button wired to ground needs a **pull-up**, so **pressed reads 0**. This is called active-low. PUPDR can turn on a pull-up inside the chip, so you do not need an extra part.



Floating



Pulled up



Pulled down

Two bits per pin: 00 floating, 01 pull-up, 10 pull-down. An internal resistor means **one fewer part** on the board. This is Chapter 1's cost argument again.

- 1 Why This Matters
- 2 Inside a GPIO Pin
- 3 **The GPIO Registers**
- 4 Experiments
- 5 Design Challenge

Register	Bits/pin	Purpose
MODER	2	Mode: input, output, alternate function, analog
OTYPER	1	Output type: push-pull or open-drain
OSPEEDR	2	Output slew rate
PUPDR	2	Pull-up, pull-down, or neither
IDR	1	Input data — read the pin
ODR	1	Output data — drive the pin
BSRR	2	Atomic bit set / reset
AFR[2]	4	Which alternate function (Chapter 10)

Bits per pin tells you how to build the mask

One bit wide: mask 1, at position n . Two bits wide: mask 3, at position $2n$. Four bits wide: mask 0xF, at position $4n$.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits	Mode
00	Input (reset state)
01	General-purpose output
10	Alternate function
11	Analog

```
GPIOA->MODER &= ~(3U << (5*2)); // clear
```

```
GPIOA->MODER |= (1U << (5*2)); // set 01
```

Clear first, then set. Without the clear step, you cannot change a 11 into a 01.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

ODR `|= x` is a **read-modify-write**: three separate steps. If an interrupt changes another pin in between, that change is **lost**.

```
GPIOA->BSRR = (1U << 5);      // PA5 on
GPIOA->BSRR = (1U << (5+16)); // PA5 off
```

One **write**, no read. Note the `=` instead of `|=`. Using `|=` here would be a bug.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	OTGHS ULPIEN	OTGHS EN	Res.	Res.	Res.	Res.	Res.	Res.	DMA2 EN	DMA1 EN	Res.	Res.	BKP SRAMEN	Res.	Res.
	rw	rw							rw	rw			rw		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	CRC EN	Res.	Res.	Res.	Res.	GPIOH EN	GPIOG EN	GPIOF EN	GPIOE EN	GIPOD EN	GPIOC EN	GPIOB EN	GPIOA EN
			rw					rw	rw	rw	rw	rw	rw	rw	rw

The classic first bug

Every peripheral starts with its clock **disabled**, to save power. A peripheral with no clock **does not respond to writes**: the writes are simply lost. No fault, no error, nothing happens.

```
RCC->AHB1ENR |= (1U << 0); // GPIOA
```

- 1 Why This Matters
- 2 Inside a GPIO Pin
- 3 The GPIO Registers
- 4 Experiments**
- 5 Design Challenge

With ODR

```
#include "stm32f4xx.h"
void delay_ms(uint32_t ms) { for (uint32_t i=0;i<ms*4000;i++)
    __NOP(); }
int main(void) {
    RCC->AHB1ENR |= (1U << 0);
    GPIOA->MODER &= ~(3U << 10);
    GPIOA->MODER |= (1U << 10);
    while (1) {
        GPIOA->ODR |= (1U << 5);
        delay_ms(500);
        GPIOA->ODR &= ~(1U << 5);
        delay_ms(500);
    }
}
```

With BSRR

```
#include "stm32f4xx.h"
void delay_ms(uint32_t ms) { for (uint32_t i=0;i<ms
    *4000;i++) __NOP(); }
int main(void) {
    RCC->AHB1ENR |= (1U << 0);
    GPIOA->MODER &= ~(3U << 10);
    GPIOA->MODER |= (1U << 10);
    while (1) {
        GPIOA->BSRR = (1U << 5);
        delay_ms(500);
        GPIOA->BSRR = (1U << 21);
        delay_ms(500);
    }
}
```

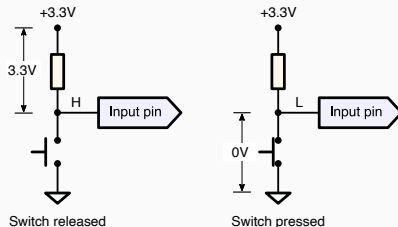
Same effect, but the second version is **atomic** and one instruction shorter.

$5 \times 2 = 10$, so PA5's mode bits sit at position 10. $5 + 16 = 21$ is its reset bit in BSRR.

Experiment 3: Button Controls LED

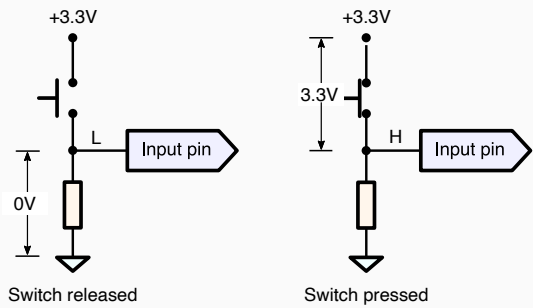
```
#include "stm32f4xx.h"
int main(void) {
    RCC->AHB1ENR |= (1U << 2) | (1U << 0); // GPIOC, GPIOA
    GPIOA->MODER &= ~(3U << 10);
    GPIOA->MODER |= (1U << 10); // PA5 output (LED)
    GPIOC->MODER &= ~(3U << (13*2)); // PC13 input
    GPIOC->PUPDR &= ~(3U << (13*2));
    GPIOC->PUPDR |= (1U << (13*2)); // pull-up

    while (1) {
        if (!(GPIOC->IDR & (1U << 13))) {
            GPIOA->BSRR = (1U << 5); // pressed -> on
        } else {
            GPIOA->BSRR = (1U << (5+16)); // released -> off
        }
    }
}
```



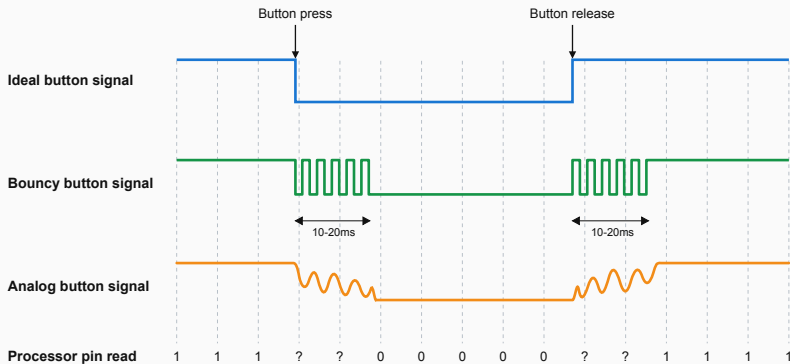
The button is **active low**: pressed reads 0. When released, the $\sim 40 \text{ k}\Omega$ pull-up holds the pin HIGH. When pressed, the switch gives a low-resistance path to ground. That path wins.

The Other Convention: Pull-Down



A pull-down with an **active-high** switch is the mirror image: released reads LOW, and pressed connects the pin to V_{DD} . Use `PUPDR = 10` instead of `01`, and change the polarity in the code to match.

Experiment 4: Why a Button Needs Debouncing



A mechanical contact does not close cleanly. It **bounces** for several milliseconds. A processor that polls at MHz speed sees one press as **dozens**.

Debounce

```
#include "stm32f4xx.h"
void delay_ms(uint32_t ms) { for (uint32_t i=0;i<ms*4000;i++)
    __NOP(); }
void handle_press(void) { /* app-specific action */ }

int main(void) {
    RCC->AHB1ENR |= (1U << 2);          // GPIOC clock
    GPIOC->MODER &= ~(3U << (13*2));    // PC13 input
    GPIOC->PUPDR &= ~(3U << (13*2));
    GPIOC->PUPDR |= (1U << (13*2));     // pull-up

    while (1) {
        if ((GPIOC->IDR & (1U << 13)) == 0) {
            delay_ms(20);                // let it settle
            if ((GPIOC->IDR & (1U << 13)) == 0) {
                // a real press
                while ((GPIOC->IDR & (1U << 13)) == 0)
                    ;                    // wait for release
                handle_press();
            }
        }
    }
}
```

Mind the brackets

In C, & binds *looser* than ==. So `IDR & (1<<13) == 0` is parsed as `IDR & ((1<<13) == 0)`, which is always zero.

Always put brackets around the mask.

A 20 ms delay is the usual choice. It is long enough for a mechanical switch to settle, and short enough that a user does not notice.

Experiment 5: Measuring Reaction Time

```
#include "stm32f4xx.h"
void delay_ms(uint32_t ms) { for (uint32_t i=0;i<ms*4000;i++) __NOP(); }
volatile uint32_t reaction_ms;    // inspect this in the debugger
int main(void) {
    RCC->AHB1ENR |= (1U << 0) | (1U << 2); // GPIOA, GPIOC
    GPIOA->MODER &= ~(3U << 10);
    GPIOA->MODER |= (1U << 10);           // PA5 output (LED)
    GPIOC->MODER &= ~(3U << (13*2));      // PC13 input (button)
    GPIOC->PUPDR &= ~(3U << (13*2));
    GPIOC->PUPDR |= (1U << (13*2));      // pull-up

    while (1) {
        GPIOA->BSRR = (1U << (5+16));    // LED off
        delay_ms(2000);                  // wait before the cue
        GPIOA->BSRR = (1U << 5);          // LED on: react now

        reaction_ms = 0;
        while (GPIOC->IDR & (1U << 13)) { // until the button goes LOW
            delay_ms(1);
            reaction_ms++;
        }
        GPIOA->BSRR = (1U << (5+16));    // acknowledge
        delay_ms(1000);
    }
}
```

This measurement is only as accurate as `delay_ms`, a calibrated software loop. Its timing shifts with compiler optimization and clock speed. **A hardware counter would make it exact.**

Experiment 6: Knight Rider LED Chaser

```
#include "stm32f4xx.h"
void delay_ms(uint32_t ms) { for (uint32_t i=0;i<ms*4000;i++) __NOP(); }

int main(void) {
    RCC->AHB1ENR |= (1U << 1);    // enable GPIOB clock
    GPIOB->MODER &= ~(0xFFFFU);   // clear PB0..PB7
    GPIOB->MODER |= (0x5555U);     // PB0..PB7 as outputs (01 each)

    int pos = 0, dir = 1;
    while (1) {
        GPIOB->BSRR = (0xFFU << 16); // all eight LEDs off
        GPIOB->BSRR = (1U << pos);    // light only the current one

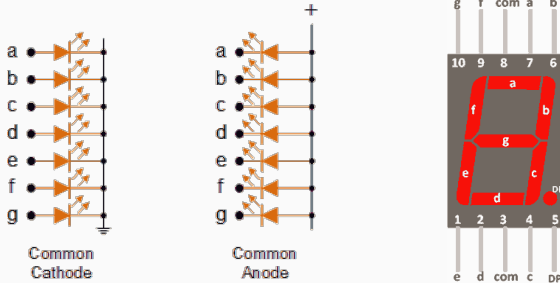
        if (pos == 7 || pos == 0) dir = -dir; // bounce at the ends
        pos += dir;

        delay_ms(80);
    }
}
```

One lit bit, walking back and forth

Only one output pin is ever high at a time; everything else is cleared first with a single **atomic** BSRR write, then the new bit is set with a second write. `pos` does the work a hardware shift register would do in an all-analogue version of this circuit — flipping `dir` at each end turns a shift into a **bounce**.

Experiment 7: A Seven-Segment Display



Use a lookup table, not per-segment logic

Deciding which of seven segments to light for each digit is a table, not an algorithm. **One array indexed by the digit** replaces a whole page of `if` statements. It is also faster and easier to check.

Common-Anode Lookup Table

```
#include "stm32f4xx.h"
void delay_ms(uint32_t ms) { for (uint32_t i=0;i<ms*4000;i++) __NOP()
    ; }

// common-anode codes, digits 0-9
// bit0=a ... bit6=g, bit7=dp -- 0 lights the segment
static const uint8_t seg7_ca[10] = {
    0xC0, 0xF9, 0xA4, 0xB0, 0x99,
    0x92, 0x82, 0xF8, 0x80, 0x90
};

int main(void) {
    RCC->AHB1ENR |= (1U << 1); // enable GPIOB clock
    GPIOB->MODER &= ~(0xFFFFU); // clear PB0..PB7
    GPIOB->MODER |= (0x5555U); // PB0..PB7 as outputs

    uint8_t digit = 0;
    while (1) {
        GPIOB->ODR = seg7_ca[digit]; // already active-low
        digit = (digit + 1) % 10;
        delay_ms(1000);
    }
}
```

Why the values look inverted

Common-anode means the shared pin is tied to VDD; each segment lights when its own pin is pulled **LOW**. So the table is the bitwise complement of the familiar common-cathode codes: 0 is 0x3F for a cathode display but 0xC0 here.

Because the table is already active-low, driving it is a single ODR write — no inversion in the loop.

- A GPIO **port** has 16 pins. Each register controls all of them, so change only the bits for your pin.
- MODER selects **input / output / alternate / analog**. It uses 2 bits per pin, so pin n sits at $2n$.
- **Push-pull** can drive both high and low. **Open-drain** can only pull low, which is what lets devices share a wire.
- ODR drives a pin, IDR reads one, and BSRR sets or resets a pin **atomically**.
- A button is usually **active-low** with a pull-up, and needs **debouncing**, since contacts bounce for a few milliseconds.
- Always **enable the port's clock** first, or writes go nowhere.

Next: Chapter 9 – Interrupts and Event-Driven Programming

Polling a button wastes the whole processor. Next, we let the hardware tell us instead.

- 1 Why This Matters
- 2 Inside a GPIO Pin
- 3 The GPIO Registers
- 4 Experiments
- 5 Design Challenge

The brief

A control panel needs eight indicator LEDs, four momentary buttons, and one output driving a 24 V relay coil through an external transistor.

Produce the complete pin-level design for the STM32F446RE: which port and pins, which MODER setting for each, push-pull or open-drain for each output, and which pull resistors for each input. Justify the relay output's configuration in particular. Then state how many register writes your design needs to initialize the whole panel, and whether any of them can be combined.

Questions?

Introduction to ARM Cortex-M Microprocessors — Chapter 8